

Spec: Image Upload Coordination with Push Notification

Goal: Ensure that UploadImageDetailsEndpoint (B) only processes an image after UploadImageEndpoint (A) has completed the upload, without polling, using a push-based mechanism.

Components:

1. Singleton Coordinator
2. Class: ImageUploadCoordinator
3. Responsibilities:
 - Track the state of uploads (TaskCompletionSource per ImageId).
 - Allow B to wait for A to finish.
 - Handle A completing before or after B arrives.

Methods: Task WaitForUploadAsync(string imageId); void MarkUploaded(string imageId);

Behavior: - WaitForUploadAsync(imageId): - If upload already completed -> returns a completed task immediately. - Else -> returns a Task that completes once MarkUploaded is called. - MarkUploaded(imageId): - Records the upload as completed. - Completes the TCS for any waiting B endpoint(s).

1. Endpoint A: UploadImageEndpoint
2. Request: UploadImageRequest
3. Actions:
 1. Save the uploaded image.
 2. Call _coordinator.MarkUploaded(imageId) to signal completion.
 3. Return HTTP 200 with confirmation.
4. Endpoint B: UploadImageDetailsEndpoint
5. Request: UploadImageDetailsRequest
6. Actions:
 1. Call await _coordinator.WaitForUploadAsync(imageId) — pauses automatically if A hasn't finished.
 2. Crop / caption / process the image.
 3. Return HTTP 200 with processing confirmation.

Flow Diagrams:

Scenario 1: B arrives first B -> WaitForUploadAsync(imageId) (task created, awaiting) A -> Upload completes A -> MarkUploaded(imageId) (task completed) B -> resumes processing

Scenario 2: A arrives first A -> Upload completes A -> MarkUploaded(imageId) (completed uploads recorded) B -> WaitForUploadAsync(imageId) (task sees upload done, returns immediately) B -> proceeds with processing

Technical Notes: - Use `ConcurrentDictionary<string, TaskCompletionSource<string>>` to track ongoing uploads. - Use `ConcurrentDictionary<string, bool>` or equivalent to record completed uploads. - `TaskCompletionSource` should use `TaskCreationOptions.RunContinuationsAsynchronously` to avoid blocking threads. - Works for multiple concurrent uploads within a single app instance. - For multi-server deployments, replace with distributed pub/sub or queue.

Outcome: - B never processes an image before A finishes uploading. - Push-based, no polling required. - Handles both out-of-order request arrival scenarios safely.